

Introduction to Object Oriented Programming (OOP) in Python

Contents

1	Before OOP	3
2	Problem with This Approach	3
3	Real-world Thinking	3
4	What is OOP?	4
5	What is a Class?	4
6	What is an Object?	4
7	Creating First Class	5
8	Creating Object	5
9	<code>__init__</code> Method	5
10	<code>self</code> Keyword	5
11	Storing Data in Object	5
12	Full Example	6
13	Creating Objects	6
14	Object Data	6
15	Methods	6
16	Grade Calculation Method	6
17	Method Call	6
18	Data Validation	7
19	Raising Exception	7

20 Why OOP is Powerful	7
21 Real-world Examples	7
22 Interesting Observation	8
23 Functions are Also Objects	8
24 Modules are Also Objects	8
25 Big Realization	8
26 More Examples	9
27 Everything is Object in Python	9
28 Final Thought	9
29 Exercises	10
29.1 Exercise 1	10
29.2 Exercise 2	10
29.3 Exercise 3	10
29.4 Exercise 4	10
29.5 Exercise 5	10
30 Challenge Questions	11
30.1 Challenge 1	11
30.2 Challenge 2	11
30.3 Challenge 3	11
30.4 Challenge 4	11
30.5 Challenge 5	11

1 Before OOP

Till now, we stored data separately like this:

```
stu1_name = "ram"
stu1_address = "ktm"
stu1_marks = 80

stu2_name = "hari"
stu2_address = "btm"
stu2_marks = 90
```

And actions separately:

```
def calculate_grade(marks):
    ...
```

2 Problem with This Approach

Imagine:

- 10 students
- 100 students
- 10,000 students

Then variables become:

```
stu100_name
stu100_address
stu100_marks
```

Very messy.

Difficult to:

- manage
- update
- scale

3 Real-world Thinking

Think about a real student.

A student has:

- data
 - name
 - address
 - marks

- actions
 - calculate grade
 - show profile
 - attend class

In programming:

OOP combines both together.

4 What is OOP?

OOP means:

Object Oriented Programming

Programming style based on:

- Objects
- Classes

5 What is a Class?

Class is:

Blueprint / Design / Template

Example:

- House map
- Car design
- Student structure

6 What is an Object?

Object is:

Real instance created from class

Example:

- Blueprint = class
- Actual house = object

7 Creating First Class

```
class Student:  
    pass
```

8 Creating Object

```
s1 = Student()
```

```
s1
```

is object.

9 __init__ Method

```
def __init__(self, name, address, marks):
```

called:

Constructor

Runs automatically when object is created.

10 self Keyword

```
self
```

means:

Current object itself

11 Storing Data in Object

```
self.name = name
```

Left side:

- object variable

Right side:

- parameter value

12 Full Example

```
class Student:

    def __init__(self, name, address, marks):

        self.name = name
        self.address = address
        self.marks = marks
```

13 Creating Objects

```
s1 = Student("ram", "ktm", 80)
s2 = Student("hari", "btm", 90)
```

14 Object Data

```
print(s1.name)
print(s2.address)
```

Each object stores:

its own data

15 Methods

Functions inside class are called:

Methods

16 Grade Calculation Method

```
def calculate_grade(self):
```

Notice:

```
self.marks
```

because marks belong to object.

17 Method Call

```
s1.calculate_grade()
```

Python automatically sends:

```
self
```

18 Data Validation

```
if marks <= 100:
```

Checks valid marks.

19 Raising Exception

```
raise Exception("Invalid marks")
```

Used when:

- invalid situation occurs

20 Why OOP is Powerful

OOP helps:

- organize code
- reduce repetition
- model real-world systems
- make large applications manageable

21 Real-world Examples

Everything around us can be modeled using OOP:

- Student
- BankAccount
- Car
- Product
- User
- Camera
- Game Character

22 Interesting Observation

```
print(type(s1))
```

Output:

```
<class '__main__.Student'>
```

Meaning:

s1 is object of Student class

23 Functions are Also Objects

```
def ex():  
    pass  
  
print(type(ex))
```

Output:

```
<class 'function'>
```

Interesting:

Functions are also objects in Python.

24 Modules are Also Objects

```
import time  
  
print(type(time))
```

Output:

```
<class 'module'>
```

Even modules are objects.

25 Big Realization

From day 1, we were already using:

- classes
- objects

without realizing it.

Example:

```
name = "ram"
```


Actually:

```
"ram"
```

is object of:

```
str
```

class.

26 More Examples

```
5
```

is object of:

```
int
```

```
[]
```

is object of:

```
list
```

```
{}
```

is object of:

```
dict
```

27 Everything is Object in Python

This is one of Python's biggest features.

Almost everything is:

Class + Object

28 Final Thought

Earlier:

- we used objects

Now:

- we can create our own objects

This is a huge step in programming journey.

You are no longer just:

“using Python”

Now you are starting to:

“design systems”

29 Exercises

29.1 Exercise 1

Create class:

```
Car
```

with:

- brand
- color
- price

29.2 Exercise 2

Create method:

```
show_info()
```

to print all details.

29.3 Exercise 3

Create 3 student objects.

29.4 Exercise 4

Add method:

```
is_passed()
```

Return:

- True if marks ≥ 40
- False otherwise

29.5 Exercise 5

Create:

```
BankAccount
```

class with:

- deposit
- withdraw
- balance check

30 Challenge Questions

30.1 Challenge 1

Create class:

```
Rectangle
```

with methods:

- area
- perimeter

30.2 Challenge 2

Create:

```
Employee
```

class with salary validation.

30.3 Challenge 3

Create login system using:

- username
- password

inside class.

30.4 Challenge 4

Create:

```
LibraryBook
```

class with:

- issue book
- return book

30.5 Challenge 5

Create:

```
ShoppingCart
```

class storing:

- products
- quantities
- total bill